



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-14 «Цифровые технологии обработки данных»

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ №10

на тему: «Настройка анимации скелета - риггинг»

Выполнил:

Студент группы БСБО-43-24

Стасенок Юлия Анатольевна

Проверил:

к. т. н., доцент

Кашкин Евгений Владимирович

Москва, 2025 г.

Оглавление

Цель работы	3
Задание	3
Ход работы.....	3
Выводы	11
Список использованных источников	12

Цель работы

Цель данного этапа работы состоит в том, чтобы настроить анимации скелета объекта.

Задание

В ходе практического задания необходимо выполнить следующие пункты:

1. изучить видеоматериал по данному заданию;
2. открыть ресурс mixamo.com и выбрать персонажа;
3. скачать персонажа;
4. подобрать необходимые анимации для персонажа;
5. загрузить модель и анимации и персонажа в Unity;
6. добавить персонажа на сцену;
7. настроить анимации для персонажа и логику их переходов в зависимости от состояний;
8. использовать Animation Event для реализации любой механики;
9. реализовать полностью рабочую машину состояний, с учётом анимаций;
10. протестировать.

Ход работы

Сначала на сайте Sketchfab.com была выбрана модель врага (см. рисунок 10.1).



Рисунок 10.1 Выбранная модель врага

Далее выбранная модель врага была загружена в mixamo.com (см. рисунок 10.2).

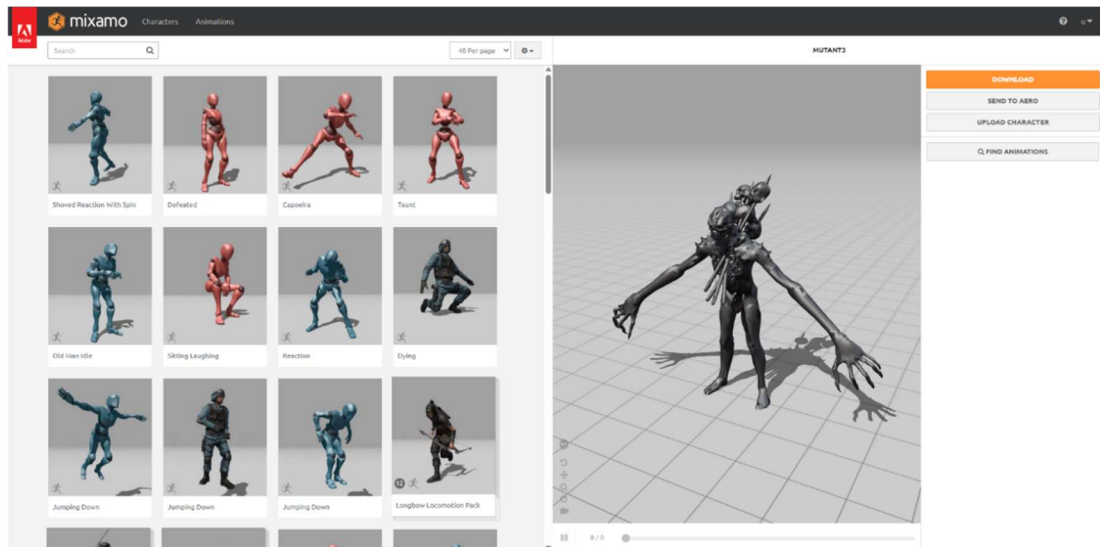


Рисунок 10.2. Выбранная модель в mixamo.com

Далее для врага необходимо было выбрать анимации ходьбы, бездействия и атаки. Были выбраны следующие анимации - Mutant Walking для реализации анимации ходьбы, Mutant Breathing Idle для реализации анимации бездействия, Standing Melee Combo Attack Ver. 2 для реализации анимации атаки.

Скриншот врага, исполняющего анимацию Mutant Walking в mixamo.com, представлен на рисунке 10.3.

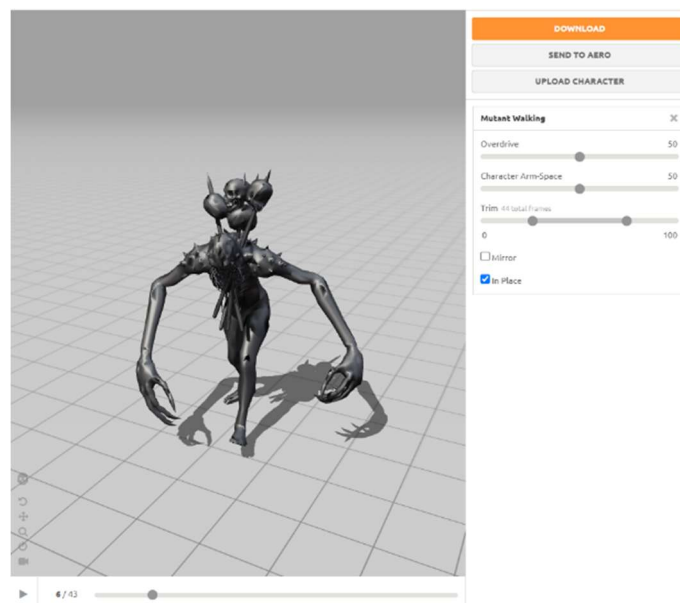


Рисунок 10.3 Анимация Mutant Walking на модели врага

Скриншот врага, исполняющего анимацию Mutant Breathing Idle в mixamo.com, представлен на рисунке 10.4.

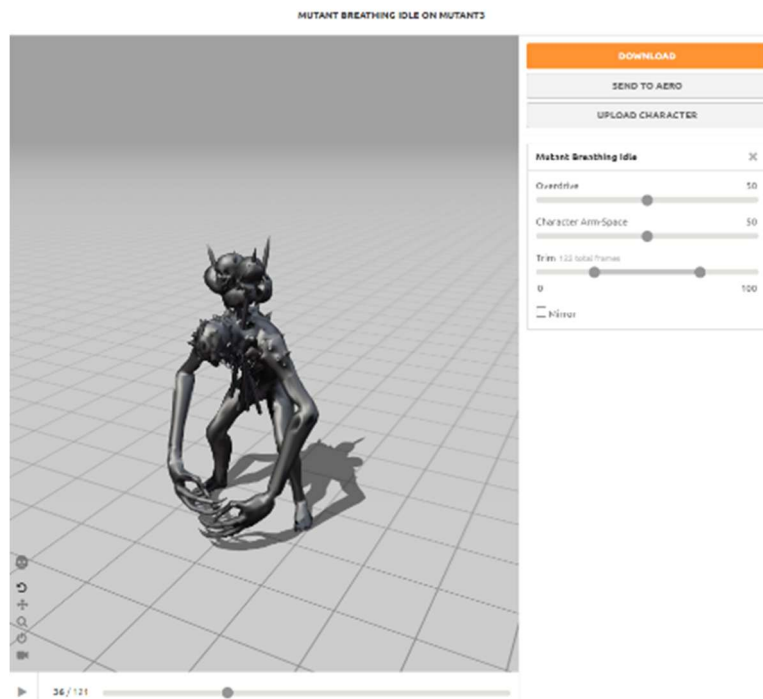


Рисунок 10.4 Анимация Mutant Breathing Idle на модели врага

Скриншот врага, исполняющего анимацию Standing Melee Combo Attack Ver.2 в mixamo.com, представлен на рисунке 10.5.

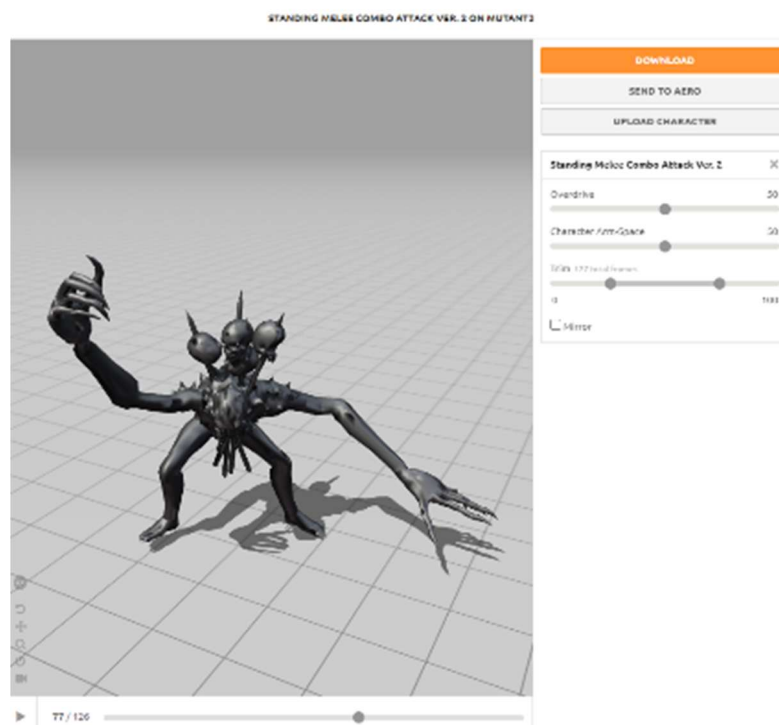


Рисунок 10.5 Анимация Standing Melee Combo Attack Ver.2 на модели врага

Далее выбранные анимации были скачаны в формате .fbx, после чего вместе с моделью врага были добавлены в проект в общую папку (см. рис.10.4).

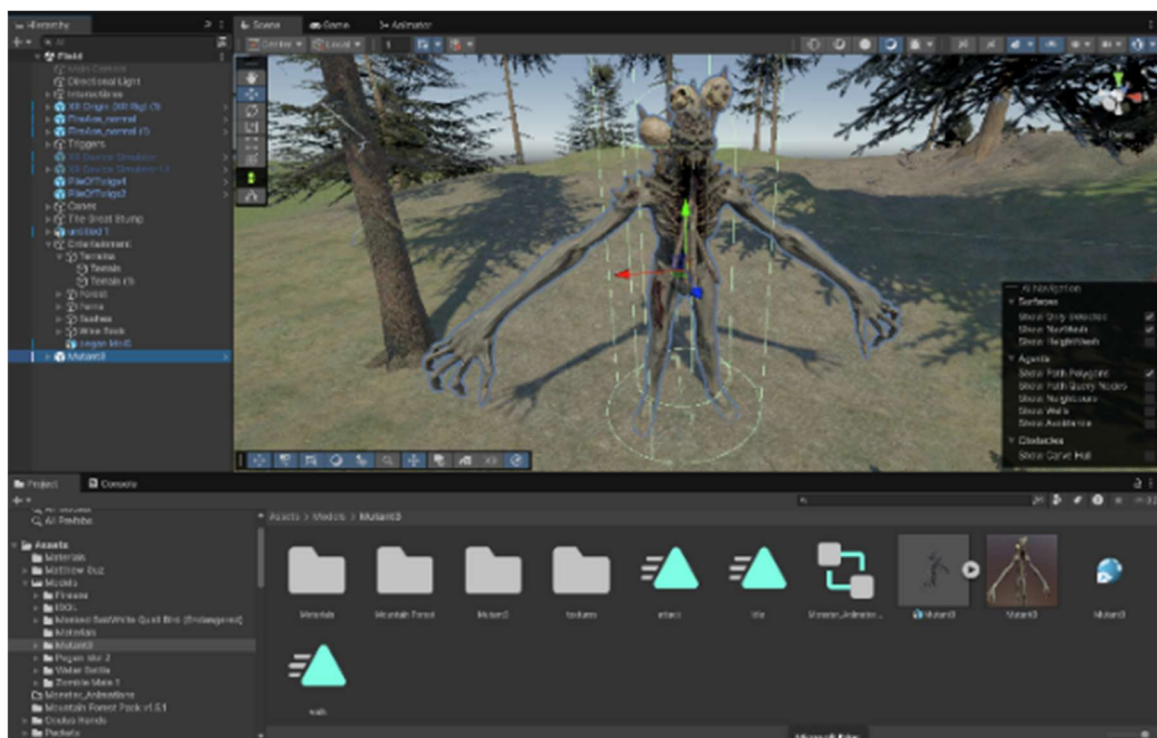


Рисунок 10.6. Модель врага и анимации в движении

Далее при помощи объекта Animator для врага была построена логика переходов в зависимости от его состояний – бездействия, ходьбы или атаки (см. рисунок 10.7).

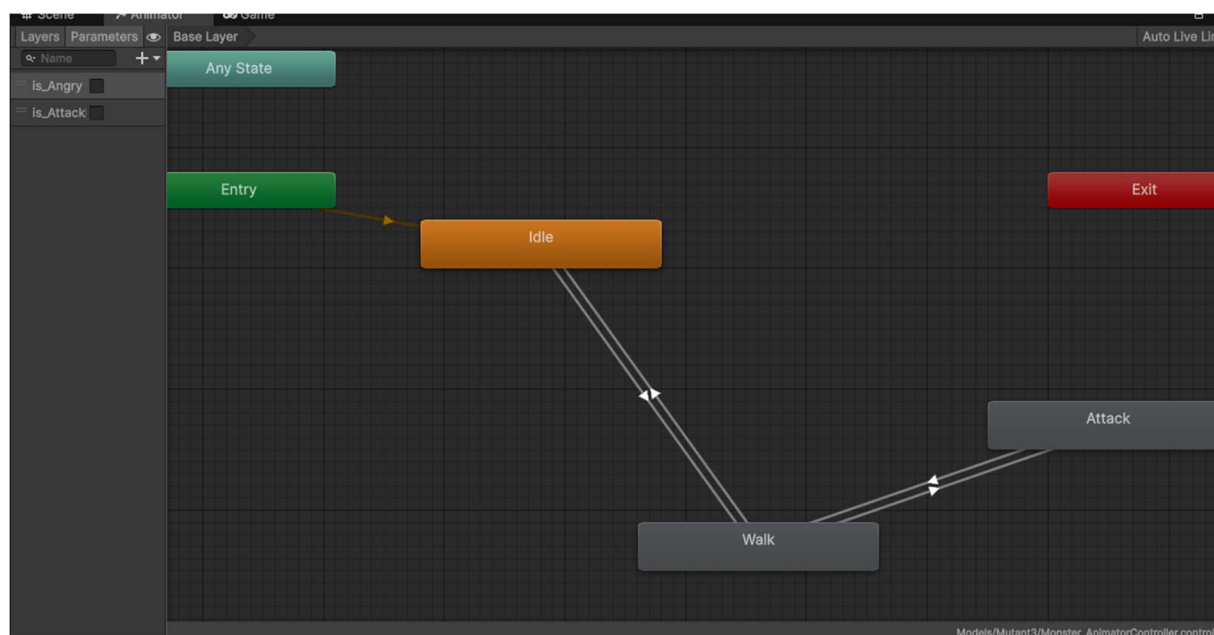


Рисунок 10.7. Логика переходов для врага

Состояние ходьбы и переходы в него из других состояний были реализовано при помощи скрипта WalkState, представленного в листинге 10.1.

```
using UnityEngine;
public class WalkState : AState{
    public override void EnterState(EnemyStateManager enemyManager)
    {enemyManager.animator.SetBool("is_Angry", true);
        enemyManager.SetSpeed(enemyManager.walkSpeed);}
    public override void ExitState(EnemyStateManager enemyManager){
        enemyManager.animator.SetBool("is_Angry", false);}
    public override void UpdateState(EnemyStateManager enemyManager){
        if (enemyManager.CheckOnTarget() > enemyManager.agroDistance){
            enemyManager.animator.SetBool("is_Angry", false);
            enemyManager.SwitchState(enemyManager.idleState);}
        if (enemyManager.CheckOnTarget() <= enemyManager.agroDistance &&
enemyManager.CheckOnTarget() <= enemyManager.attackDistance){
            enemyManager.animator.SetBool("is_Angry", true);
            enemyManager.animator.SetBool("is_Attacking", true);
            enemyManager.SwitchState(enemyManager.attackState);}
        if (enemyManager.CheckOnTarget() <= enemyManager.agroDistance &&
enemyManager.CheckOnTarget() > enemyManager.attackDistance){
            enemyManager.animator.SetBool("is_Angry", true);
            enemyManager.SwitchState(enemyManager.walkState);}}}
```

Листинг 10.1. Скрипт WalkState

Состояние бездействия врага и переходы в это состояние из других были реализованы при помощи скрипта IdleState, данный скрипт представлен в листинге 10.2.

```
using UnityEngine;
public class IdleState : AState
{
    public override void EnterState(EnemyStateManager enemyManager) {
        enemyManager.SetSpeed(0);
        enemyManager.animator.SetBool("is_Attacking", false);
        enemyManager.animator.SetBool("is_Angry", false);}
    public override void ExitState(EnemyStateManager enemyManager)
    { }
    public override void UpdateState(EnemyStateManager enemyManager) {
        if (enemyManager.CheckOnTarget() < enemyManager.agroDistance) {
            enemyManager.SwitchState(enemyManager.walkState);}}}
```

10.2. Скрипт IdleState

Состояние атаки врага и переходы в него из других состояний были реализованы при помощи скрипта AttackState, данный скрипт представлен в листинге 10.3. Также в этом скрипте реализована механика, позволяющая врагу поворачиваться в сторону игрока, не выходя из состояния атаки.

```
using UnityEngine;

public class AttackState : AState
{
    public override void EnterState(EnemyStateManager enemyManager)
    {
        enemyManager.SetSpeed(0);
        enemyManager.animator.SetBool("is_Attacking", true);
    }

    public override void ExitState(EnemyStateManager enemyManager)
    {
        enemyManager.animator.SetBool("is_Attacking", false);
    }

    public override void UpdateState(EnemyStateManager enemyManager)
    {
        if (enemyManager.CheckOnTarget() > enemyManager.attackDistance)
        {
            enemyManager.animator.SetBool("is_Angry", true);
            enemyManager.animator.SetBool("is_Attacking", false);
            enemyManager.SwitchState(enemyManager.walkState);
        }
        else if (enemyManager.CheckOnTarget() <=
enemyManager.attackDistance)
        {
            enemyManager.animator.SetBool("is_Angry", true);
            enemyManager.animator.SetBool("is_Attacking", true);
            enemyManager.SwitchState(enemyManager.attackState);
            enemyManager.RotateTowards(enemyManager.CheckOnTargetRotation());
        }
    }
}
```

Листинг 10.3. Скрипт AttackState

Далее с помощью компонента Animator были реализованы активация и отключение коллайдеров-триггеров на руках врага (см. листинг 10.4).

```
using UnityEngine;

public class AnimationEventSripts : MonoBehaviour{
    [SerializeField] public Animator animator1;
    [SerializeField] EnemyStateManager enemyStateManager;
    [SerializeField] AttackState attackState;
    [SerializeField] Collider damager1;
    [SerializeField] Collider damager2;
    private void Awake(){
        enemyStateManager = this.GetComponent<EnemyStateManager>();
    }
    public void OnOffDamager1(int onoff){
        if (onoff == 2)
            {damager1.enabled = false;}
        else if (onoff == 1)
            {damager1.enabled = true;}}
    public void OnOffDamager2(int onoff)
    {
        if (onoff == 2)
            {damager2.enabled = false;}
        else if (onoff == 1)
            {damager2.enabled = true;
        }
    }
}
```

Листинг 10.4. Скрипт активации и отключения коллайдеров-триггеров

С помощью этих коллайдеров-триггеров и метода OnTriggerEnter была реализована механика получения урона игроком (см. листинг 10.5).

```
using UnityEngine;

public class PlayerHP : MonoBehaviour
{
    int hp = 100;
    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.TryGetComponent(out EnemyDamager enemyDamager))
            {this.hp -= enemyDamager.enemyDmg;}
    }
}
```

Листинг 10.5. Скрипт получения урона игроком

Отключение и активация коллайдеров-триггеров были реализованы при помощи AnimationEvent для анимации атаки. При помощи AnimationEvent в момент, когда враг проигрывает определенные кадры анимации атаки, вызываются функции, активирующие и отключающие коллайдеры-триггеры (см. рисунок 10.8).

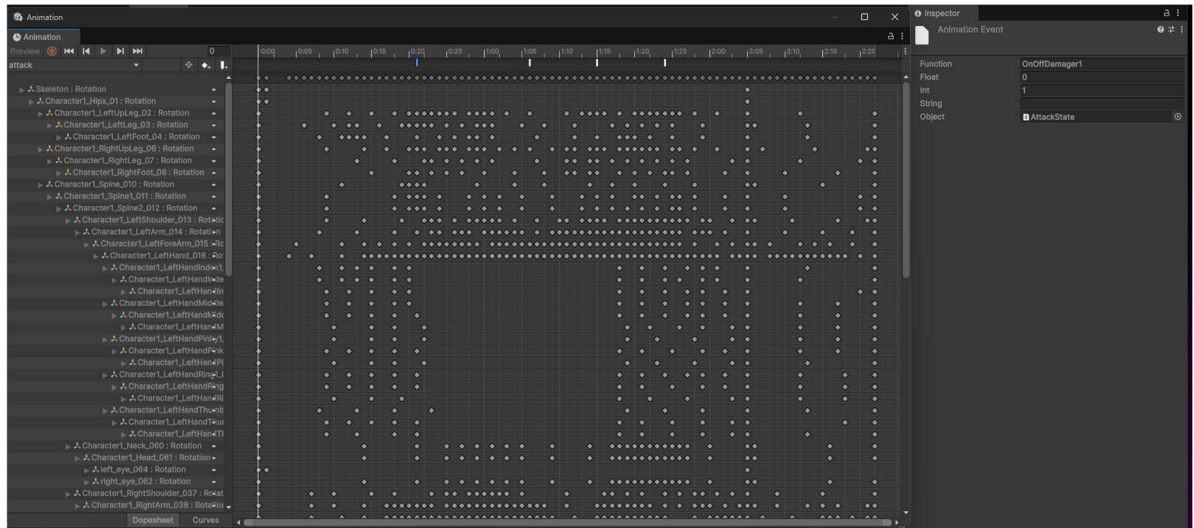


Рисунок 10.8. AnimationEvent для анимации атаки врага

Далее была проведена тестировка реализованных механик с целью проверки их работоспособности.

Скриншот врага, идущего за игроком представлен на рисунке 10.9.



Рисунок 10.9. Скриншот врага, бегущего за игроком

На рисунке 10.10 представлен скриншот врага, атакующего игрока.

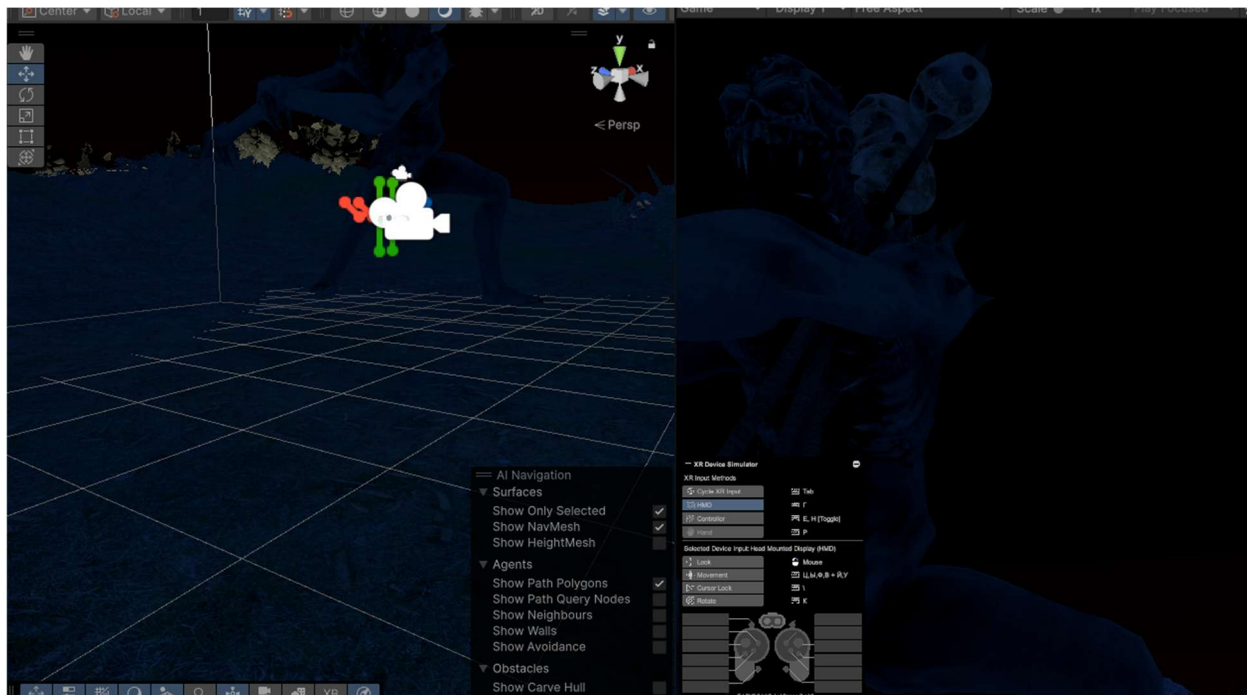


Рисунок 10.10. Скриншот врага, атакующего игрока

Выводы

В ходе работы были выбраны модель для врага и анимации ходьбы, атаки и бездействия. После чего они были загружены в движок и добавлены на сцену. Потом была настроена логика переходов между всеми состояниями врага – состоянием бездействия, ходьбы и атаки. Все состояния далее были реализованы при помощи соответствующих скриптов. Далее при помощи Animation Event была реализована активация коллайдеров-триггеров для нанесения урона игроку. Затем была проведена тестировка реализованных механик.

Список использованных источников

1. Руководство: Использование событий в анимации [Электронный ресурс]
URL: <https://docs.unity3d.com/ru/530/Manual/animeditor-AnimationEvents.html> (дата обращения: 15.04.2025).